

Project Proposal

RL-PQC-VPN: A Reinforcement Learning Driven Post-Quantum Cryptographic VPN System

Version	2: revised to match the implementation (Week 4 checkpoint, September 2026)
Supersedes	RL_PQC_VPN_Proposal.pdf (v1)
Platform	Linux desktop client (Ubuntu 22.04 LTS or later); self-hosted Ubuntu 24.04 server

1. Introduction

Most VPNs today rely on classical cryptography, specifically Diffie-Hellman key exchange and elliptic curve algorithms. These are mathematically secure against classical computers, but a sufficiently powerful quantum computer running Shor's algorithm can break them. The NSA, NIST, and multiple governments have already begun mandating a transition to quantum-resistant cryptography.

The threat that makes this urgent right now is called Harvest Now, Decrypt Later (HNDL). Attackers are already recording encrypted VPN traffic, storing it, and waiting for quantum hardware to mature so they can decrypt it retroactively. Data sent over a classical VPN today could be readable within a decade.

This project builds a VPN that addresses this threat by adding NIST-standardised post-quantum key exchange to the tunnel, and embedding a reinforcement learning agent that learns the device's behaviour and makes intelligent, real-time decisions about which algorithm to use and when to rotate session keys. No existing VPN product combines these two capabilities.

2. Problem Statement

Current post-quantum VPN implementations treat quantum resistance as a binary switch: either on at a fixed security level, or off. NordVPN and ExpressVPN both launched PQC support in 2024–2025, but both apply it statically. There is no adaptation based on device conditions, traffic sensitivity, or detected threats. This creates three gaps:

- Static PQC is wasteful on low-powered devices and mobile hardware, where running ML-KEM-1024 continuously drains battery and CPU without justification.
- Static PQC has no response mechanism for operational threats: key compromise through memory access, session key ageing, or network anomalies that indicate a man-in-the-middle.
- No existing system uses machine-local reinforcement learning to learn per-device security policies.

This system addresses all three gaps.

3. Proposed Solution

We are building a complete desktop VPN application with four integrated components: a post-quantum encrypted tunnel, a reinforcement learning decision engine, a three-layer anomaly detection pipeline, and

a native desktop application built with Tauri. The primary development and deployment platform is Linux. Windows is planned as a second phase.

All client logic lives in one shared Rust library, `vpn_core`: cryptography, the handshake protocol client, RL inference and anomaly detection. `vpn_core` never calls an operating-system API directly. Every platform fact (CPU load, network readings, the local WireGuard interface) enters through a trait that the desktop app implements. A future mobile app would implement the same traits and reuse the whole core unchanged. The three team members work on clearly separated layers that connect through frozen, versioned contracts.

3.1 Post-Quantum VPN Tunnel

The VPN tunnel is built on WireGuard [2], a fast and heavily audited modern VPN protocol. WireGuard intentionally avoids protocol agility, so we do not replace its Diffie-Hellman handshake directly. Instead, we use WireGuard's built-in Pre-Shared Key (PSK) feature: both peers derive a quantum-safe shared secret and inject it as the WireGuard PSK, which is mixed into WireGuard's handshake [1]. Mullvad and Tailscale use this approach in production.

To remain secure against both classical and quantum adversaries, the key exchange is **hybrid**: X25519 (classical Diffie-Hellman) combined with ML-KEM (post-quantum), with the hybrid secret computed as SHA-256(X25519 secret || ML-KEM secret). The session stays secure even if one of the two algorithms is later broken.

Handshake protocol. The PQC handshake is a small, versioned TCP protocol (port 51821) that runs next to WireGuard (UDP 51820):

1. **ClientHello.** The client generates fresh ephemeral X25519 and ML-KEM keypairs and sends both public keys, the requested ML-KEM level, a random nonce and its WireGuard public key.
2. **ServerHello.** The server encapsulates to the client's ML-KEM key, adds its own ephemeral X25519 key and nonce, and signs the **entire handshake transcript** with its long-term ML-DSA-65 identity key (NIST FIPS 204). The transcript covers the protocol label, the algorithm, both nonces, both X25519 public keys, the ML-KEM public key and the ciphertext.
3. **Verification.** The client verifies the signature against the server key **pinned in the application** before deriving anything. A man-in-the-middle can relay or substitute key material but cannot sign a modified transcript.
4. **Key schedule.** Both sides run HKDF-SHA256 [12] over the hybrid secret, salted with both nonces, to derive two independent keys: the WireGuard PSK and a confirmation key.
5. **Finish.** HMAC-SHA256 tags in ClientFinish and ServerFinish prove both sides derived the same keys. The server installs the PSK with `wg set` and returns its WireGuard public key, the client's tunnel address and its port, so the client needs no out-of-band configuration.

A **rekey** re-runs this exchange for an existing session and swaps the PSK without dropping the tunnel. It may move to the same or a stronger ML-KEM level, but never a weaker one.

Tunnel traffic itself is encrypted by WireGuard using ChaCha20-Poly1305.

VPN server: a self-hosted Ubuntu VPS (DigitalOcean) runs WireGuard alongside our PQC handshake service. The service is written in Rust on the same `vpn_core` cryptography as the client, so both sides

share one implementation, and it is deployed as a hardened Docker container. It tracks multiple peers, persists their session metadata (never PSKs), and reconciles with the live WireGuard interface on restart.

3.2 Reinforcement Learning Agent

The RL agent is a PPO (Proximal Policy Optimisation) policy [4] with two hidden layers of 64 neurons each. The deployed policy network has exactly 4,932 parameters (about 20 KB), which is pinned by a test. It is trained offline with Stable-Baselines3 [3] on a custom Gymnasium environment, exported to ONNX [13], and shipped with the application. On the device, `vpn_core` runs inference with ONNX Runtime every five seconds. There is no on-device training.

The agent observes seven state variables, each normalised to a 0–1 range: (1) CPU load, (2) available RAM, (3) network latency to the VPN server, (4) upload traffic volume, (5) connection type (wired/Wi-Fi/cellular), (6) time elapsed since the last key exchange, and (7) a threat score produced by the anomaly detection pipeline. Based on this state vector, the agent selects one of four actions: apply ML-KEM-512, apply ML-KEM-768, apply ML-KEM-1024, or trigger an immediate key re-exchange. The state vector, the action space and the ONNX input/output shapes are a frozen contract shared by all team members.

The reason for dynamic rekeying is operational. Session keys sit in device RAM and can be extracted through memory attacks or side channels. Rekeying limits how much traffic is exposed if a key is ever compromised.

Decision gate. The policy is a pure function of noisy sensor readings, and every change of algorithm costs a full handshake. The client therefore acts on the policy through a debounce: a new algorithm must win for three consecutive ticks and beat the current one by a probability margin of 0.15. In simulation this halves the handshake rate (71 → 34.5 per client-hour) at the cost of a median 10-second delay on legitimate escalations. When the policy requests a rekey, the handshake runs at the stronger of the current algorithm and the policy's top-ranked ML-KEM level.

Training so far. Reward-design bugs and a discount-factor problem were found and fixed through ablations. The promoted policy scores a mean episode return 10.1% above a rule-based baseline and 24% above static ML-KEM-768 **in the training simulator**. The simulator uses hand-designed reward weights, so these are not yet measurements on real traffic. The final evaluation (Section 8) closes that gap.

3.3 Anomaly Detection

The threat score is computed in layers, added incrementally based on available device resources. The CPU reading used for gating is the same one the state pipeline collects for the RL agent.

- **Layer 1: always active (all devices).** Statistical Z-score baseline. Rolling averages of latency, packet rate and packet size are maintained, and values deviating beyond three standard deviations are flagged.
- **Layer 2: active when CPU usage is below 70%.** Five rule-based signatures, each an independent stateful check over the rolling 5-second window: port scan (destination-port fan-out with a high failure ratio); sustained retransmission spike; MitM latency signature (latency jitter from a relay hop); upload-heavy bandwidth asymmetry indicating exfiltration; and DNS server change. The 70% threshold follows systems literature showing non-linear latency increases above this level [5].

- **Layer 3: active when CPU usage is below 40%.** An Isolation Forest model [6] trained on normal traffic, detecting multivariate anomalies that individual metrics miss. It is exported for Rust inference.

A combiner takes the **maximum** score across the active layers, so one confirmed signature is not diluted by quiet ones, and the RL agent never needs to know which layers are active. The pipeline runs once every five seconds on aggregated metrics, not per packet, so it cannot become a bottleneck on the tunnel it protects.

3.4 Desktop Application: Tauri

The user interface is a native desktop application built with Tauri, which uses the operating system's native webview instead of bundling a browser engine. That typically gives a much smaller installer and much lower idle memory than Electron, which matters for a security application running alongside WireGuard.

The application has two layers:

- **Rust backend.** The Tauri backend links `vpn_core` directly, in the same process. It implements the platform traits (device readings through the `sysinfo` crate; WireGuard control), runs the 5-second agent loop, and manages the window, system tray and OS notifications. There is no separate daemon process and no local HTTP or WebSocket server, which removes an attack surface and an inter-process boundary.
- **React + TypeScript frontend.** Runs inside the webview and receives state through Tauri commands and events.

The dashboard displays:

- the current connection status
- the algorithm the agent selected
- live system metrics (CPU, RAM, latency)
- a chart of recent agent decisions
- the active anomaly detection layers
- time until the next scheduled rekey
- a session log

Platform delivery: `cargo tauri build` produces a `.deb` package and an AppImage for Linux. A Windows installer can be built from the same codebase in the second phase.

4. Cryptographic Algorithms Used

Algorithm	Standard	Role in the System
ML-KEM-512	NIST FIPS 203 [10]	Low-load sessions: lightweight PQ protection
ML-KEM-768	NIST FIPS 203 [10]	Balanced security and performance
ML-KEM-1024	NIST FIPS 203 [10]	High-sensitivity sessions, elevated threat
ML-DSA-65	NIST FIPS 204 [11]	Server identity: signs the handshake transcript, preventing MitM
X25519	RFC 7748	Classical half of the hybrid key exchange

Algorithm	Standard	Role in the System
SHA-256	FIPS 180-4	Combines the X25519 and ML-KEM secrets into the hybrid secret
HKDF-SHA256	RFC 5869 [12]	Derives the WireGuard PSK and the confirmation key
HMAC-SHA256	RFC 2104	Handshake finish tags (key confirmation)
ChaCha20-Poly1305	RFC 8439	WireGuard tunnel traffic encryption
PPO	Schulman et al. 2017 [4]	RL algorithm: policy-gradient agent for algorithm selection

HQC-256 is defined in the algorithm registry but disabled until its NIST standard is final. Enabling it later requires only re-exporting the model, because the client reads the action count from the model.

5. How the System Works: Full Flow

The user opens the application from their Applications menu. Tauri opens the window, and the Rust backend loads the trained ONNX policy and the pinned server identity key through `vpn_core`. The dashboard shows a disconnected state.

When the user clicks Connect, the state pipeline takes a reading: CPU load, available RAM, network latency, upload rate and connection type. The anomaly pipeline scores the same tick across whichever layers the CPU gate allows. The seven normalised values go to the RL policy, and its choice passes through the decision gate.

`vpn_core` then runs the hybrid X25519 + ML-KEM handshake with the server at the chosen level. It verifies the server's ML-DSA-65 signature over the transcript, derives the PSK with HKDF, and confirms it with the finish tags. The server installs the PSK with `wg set`, and the client brings its WireGuard interface up using the tunnel parameters from `ServerFinish`. The tray icon turns green.

From then on the agent loop runs every five seconds. If conditions change (the user moves to public Wi-Fi, latency starts jittering, a port scan appears), the threat score and state vector change, and the agent may raise the algorithm or trigger a rekey. A rekey swaps the PSK on the live tunnel without dropping it. The dashboard updates in real time, and escalating to ML-KEM-1024 raises a native OS notification.

When the user disconnects or quits, the tunnel is torn down and all key material is overwritten in memory (zeroizing key types throughout `vpn_core`). The trained policy is a fixed artifact, so nothing learned is written to disk.

6. Technology Stack

Desktop application

- **Tauri 2:** native desktop framework using the OS webview (WebKitGTK on Linux, WebView2 on Windows).
- **Rust:** Tauri backend. Links `vpn_core`, implements the platform traits with `sysinfo`, and manages the tray, window, notifications and agent loop.
- **React + TypeScript:** frontend UI inside the webview.
- **Tailwind CSS and Recharts:** dashboard styling and live decision/metric charts.

Shared core: `vpn_core` (Rust)

- `ml-kem`, `ml-dsa`, `x25519-dalek`: pure-Rust ML-KEM-512/768/1024, ML-DSA-65 and X25519. No C dependency, and the client and server use the same implementation.
- `sha2`, `hkdf`, `zeroize`: hybrid secret, key schedule, and memory wiping of key material.
- `ort` (ONNX Runtime): on-device policy inference.
- **uniffi interface definition** (`mobile-bindings/`): scaffolded so a mobile app could reuse the core later. Not shipped this term.

Server

- **WireGuard**: tunnel, with PSK injection through `wg set`.
- **Rust handshake service**: built on `vpn_core`. Multi-peer registry, capacity limits, startup reconciliation, replay protection and rate limiting.
- **Docker + Docker Compose**: hardened container (host networking, only `CAP_NET_ADMIN`, read-only filesystem) on a DigitalOcean Ubuntu 24.04 VPS.

Training and evaluation (Python 3.11, offline)

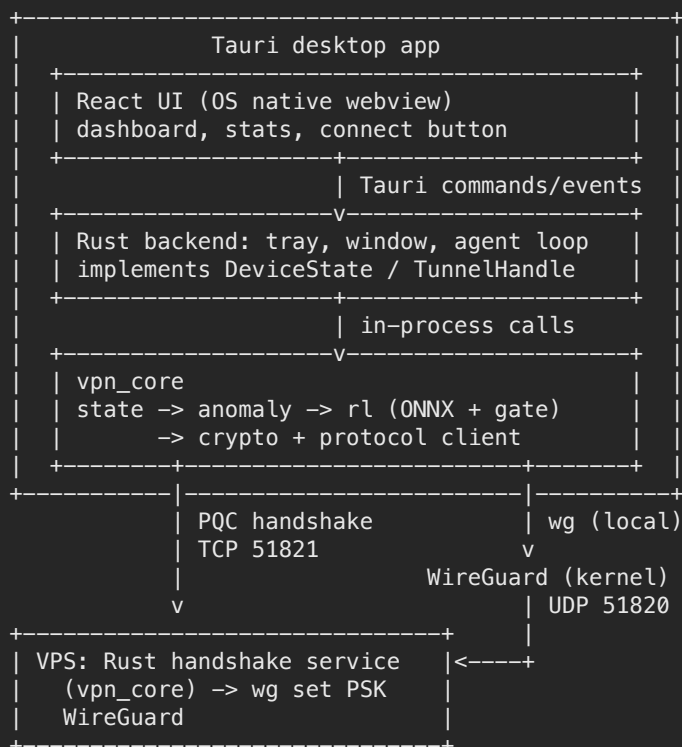
- **Stable-Baselines3 + PyTorch**: PPO training, default MLP policy (64×64).
- **Gymnasium**: custom MDP environment for the VPN decision problem.
- **ONNX / onnxruntime**: policy export and PyTorch-to-ONNX parity checks.
- **NumPy, scikit-learn (+ skl2onnx)**: Z-score reference implementation; Isolation Forest training and export.
- **psutil**: live metrics for the Python reference pipeline and demo.

Engineering practice

- **GitHub**: one branch per team member; pull requests to merge into main.
- **pytest and cargo test**: unit and integration testing. All tests must pass before a phase is closed.
- **Contract test vectors**: generated from the reference implementations (policy logits, decision-gate sequences, handshake bytes) so the Python, Rust client and Rust server provably agree byte for byte.

Architecture

USER DOWNLOADS: rl-pqc-vpn.deb / AppImage (Linux)
rl-pqc-vpn.exe (Windows, second phase)



7. What Makes This Novel

Post-quantum VPNs exist. Adaptive security systems exist. Reinforcement learning for network optimisation exists. What does not exist is a VPN client where a per-device RL agent dynamically selects between NIST-standardised PQC algorithms, and decides when to rotate keys, based on real-time system state and detected threats.

The closest published works are a 2026 paper on context-aware PQC selection for vehicular networks [7], a 2025 paper on Q-learning driven adaptive encryption for wireless sensor networks [8], and Microsoft Research's PQC-VPN work, which applies post-quantum cryptography statically [9]. None of these is a VPN client with a reinforcement learning policy. That is the gap this project fills.

The academic contribution is a formally defined Markov Decision Process (MDP) for VPN security policy optimisation:

- The state space captures device and network conditions.
- The action space covers NIST-approved algorithm choices and rekey decisions.
- The reward function balances cryptographic security strength against computational overhead.

The trained agent is evaluated against three baselines: classical WireGuard, static ML-KEM-768, and a rule-based policy.

8. Expected Outcomes

- A native desktop VPN application (Tauri) installable on Linux as a `.deb` package and AppImage, with a Windows installer as a second-phase deliverable.
- A working post-quantum key exchange using hybrid X25519 + ML-KEM on real WireGuard tunnels through PSK injection, authenticated with ML-DSA-65, specified as a versioned wire protocol.
- A trained PPO RL agent that demonstrably outperforms static algorithm selection and a rule-based policy across at least three evaluation scenarios (idle, elevated threat, resource-constrained), measured on the real system as well as in simulation.
- Anomaly detection that correctly identifies at least four threat patterns in network traffic across three device capability profiles.
- A live dashboard inside the desktop application showing RL decisions, system metrics, active anomaly layers and key rotation events.
- A comparative evaluation measuring connection latency overhead, CPU usage and security level against the three baselines.
- A shared Rust core with no platform-specific code, verified by a final review, keeping a future mobile client a thin shell rather than a rewrite.
- A full project report covering system design, the handshake protocol, the MDP formulation, training methodology, application architecture and evaluation results.

9. Changes from Version 1

Area	v1 said	v2 (matches the implementation)	Why
Client architecture	Python daemon as a Tauri subprocess; FastAPI + WebSocket on localhost:8000	Rust <code>vpn_core</code> linked into the Tauri app, in the same process	No local API attack surface or process boundary; the same core can serve a future mobile app
PQC libraries	liboqs, liboqs-python, PyCA	Pure-Rust <code>ml-kem</code> , <code>ml-dsa</code> , <code>x25519-dalek</code> , shared by client and server	No C dependency; one implementation on both sides
Server authentication	"The server signs its ML-KEM public key"	The client holds the ephemeral ML-KEM keypair; the server signs the full transcript, nonces included	Strictly stronger binding
Handshake and keys	Hybrid secret injected directly as the PSK	Versioned TCP protocol; HKDF-SHA256 key schedule; HMAC key confirmation; rekey without dropping the tunnel	Key separation, explicit key confirmation, a checkable specification
RL deployment	Fine-tuned on device; saves an experience buffer	Offline training; ONNX inference on device; decision gate	PPO does not use a replay buffer; deterministic, verifiable behaviour on device
Anomaly Layer 2	Four signature types	Five signatures + a max combiner	Bandwidth exfiltration signature added
Tooling	wgconfig; pytest only	<code>wg set</code> from Rust; <code>pytest + cargo test</code> + contract test vectors	Matches the Rust server and the cross-language contracts

Area	v1 said	v2 (matches the implementation)	Why
References	Included SP 800-38D (GCM)	Removed (AES-GCM is not used); added HKDF, ONNX; WireGuard paper now cited	Citations match what the system uses

References

1. Hülsing, A., Ning, K.-C., Schwabe, P., Weber, F., & Zimmermann, P. R. (2021). Post-Quantum WireGuard. *IEEE Symposium on Security and Privacy (SP)*, pp. 304–321. <https://doi.org/10.1109/SP40001.2021.00030>
2. Donenfeld, J. A. (2017). WireGuard: Next Generation Kernel Network Tunnel. *NDSS Symposium*. <https://www.ndss-symposium.org/ndss2017/>
3. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268), 1–8. <https://jmlr.org/papers/v22/20-1364.html>
4. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*. <https://arxiv.org/abs/1707.06347>
5. Urgaonkar, B., Shenoy, P., & Roscoe, T. (2002). Resource Overbooking and Application Profiling in Shared Internet Hosting Platforms. *USENIX OSDI*.
6. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *IEEE ICDM*, pp. 413–422. <https://doi.org/10.1109/ICDM.2008.17>
7. Alnahawi, N., et al. (2026). Context-Aware Adaptive Post-Quantum Cryptography for 6G V2X Communications. *arXiv:2602.01342*. <https://arxiv.org/abs/2602.01342>
8. Youssef, A., et al. (2025). Reinforcement Learning Q-Learning Adaptive Encryption in Wireless Sensor Networks. *MDPI Sensors*, 25(7), 2056. <https://doi.org/10.3390/s25072056>
9. Microsoft Research. Post-Quantum Cryptography VPN. <https://www.microsoft.com/en-us/research/project/post-quantum-crypto-vpn/>
10. NIST. (2024). FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard. <https://csrc.nist.gov/pubs/fips/203/final>
11. NIST. (2024). FIPS 204: Module-Lattice-Based Digital Signature Standard. <https://csrc.nist.gov/pubs/fips/204/final>
12. Krawczyk, H., & Eronen, P. (2010). HMAC-based Extract-and-Expand Key Derivation Function (HKDF). *RFC 5869*. <https://www.rfc-editor.org/rfc/rfc5869>
13. ONNX: Open Neural Network Exchange. <https://onnx.ai/>